# Secondo Progetto Machine Learning
# Approfondimento Individuale Percorsi di Eccellenza

Niccolò Sfregola

14/10/2024

**Sommario:**
Il progetto ha lo scopo di indagare le transizioni di fase quantistiche in modelli di Ising bidimensionali soggetti a campi magnetici locali randomici utilizzando tecniche avanzate di Deep Learning: verrà in particolare usata l'*Adversarial Domain Adaptation*, una metodologia che sfrutta la conoscenza del diagramma di fase nel caso in cui il campo magnetico sia nullo per prevedere le transizioni di fase in condizioni di campo magnetico non nullo. Questo approccio permette alla rete neurale di sviluppare rappresentazioni di feature invarianti tra i due domini (campo magnetico nullo e non nullo), migliorando così la capacità di generalizzazione del modello.
Per scopi comparativi, viene analizzato lo stesso problema utilizzando una rete neurale convoluzionale più semplice, ottenendo risultati compatibili rispetto a quelli ottenuti con l'approccio proposto.

## 1 Introduzione

Il modello di Ising descrive un reticolo bidimensionale di spin che possono assumere due stati (+1 o −1) e interagiscono con i loro vicini. Le transizioni di fase nel modello avvengono quando il sistema passa da una fase ordinata (con spin allineati) a una disordinata (con spin orientati in modo casuale), con una temperatura critica nota in assenza di campo magnetico.

Quando si introduce un **campo magnetico locale casuale**, il comportamento del sistema diventa più complesso, rendendo difficile identificare analiticamente le transizioni di fase.

Per affrontare questo tipo di problemi, nel quale si ha un set di dati di cui si conoscono le label e uno del quale le vogliamo predire, una possibile strada è utilizzare il metodo di *domain adaptation* [Huembeli et al., 2018], una branca del *transfer learning* : questo approccio prevede l'addestramento di un modello capace di adattarsi a distribuzioni di dati prive di etichette conosciute, sfruttando invece dati per i quali le etichette sono disponibili.

Si possono dunque distinguere due tipi di dataset (chiamati **domini**):

- Uno dei quali si conoscono le label, che verrà chiamato *dominio di sorgente* $\mathcal{S} = \{(x_s, y_s)\}$ al quale viene associata una label $d_i = 0$;
- Uno dei quali non si conoscono, che verrà chiamato *dominio target* $\mathcal{T} = \{x_t\}$ al quale viene associata una label $d_i = 1$.

Una rete di Domain Adaptation consiste in tre parti principali[1]:

- **Un feature extractor** $G_f(\mathbf{x}, \Omega_f)$, che estrae il vettore delle feature dai dati $\mathbf{f}$: in questo problema verrà usata una rete neurale convoluzionale per estrarre le feature;
- **Un predittore di label** $G_y(\mathbf{x}, \Omega_y)$,che agisce sul vettore delle feature $\mathbf{f}$;
- **Un predittore di dominio** $G_d(\mathbf{x}, \Omega_d)$, che agisce sul vettore $\mathbf{f}$, cercando di distuingere se il dataset appena analizzato appartiene al dominio sorgente oppure al dominio target.

L'obiettivo è dunque quello di predirre le label del dominio target $y_t$, allenando il modello con i dati appartenenti a entrambi i domini $x \in \mathcal{S} \cup \mathcal{T}$.

La rete viene allenata minimizzando la loss del predittore di label $L_y$ (calcolabile solo per il dominio sorgente) e massimizzando quella del predittore di dominio $L_d$ (calcolabile da entrambi i domini): in questo modo il modello è in grado di trovare feature invarianti rispetto ai due domini e quindi riesce a generalizzare la ricerca delle transizioni di fase per il dominio target.

$$(\Omega_f, \Omega_y) = \arg\min_{\Omega_f, \Omega_y} \{L_y(\Omega_f, \Omega_y) - \alpha \cdot L_d(\Omega_f, \Omega_d)\}$$
$$(\Omega_d) = \arg\max_{\Omega_d} \{L_y(\Omega_f, \Omega_y) - \alpha \cdot L_d(\Omega_f, \Omega_d)\}$$

Questo viene svolto attraverso la definizione di un layer *Gradient reversal*: quando la rete deve aggiornare i pesi, esso aggiunge una costante negativa alla back-propagation del predittore di dominio.

$$\Omega_f \leftarrow \Omega_f - \mu\Big(\frac{\partial L_y}{\partial \Omega_f} - \alpha \cdot \frac{\partial L_d}{\partial \Omega_f}\Big)$$
$$\Omega_y \leftarrow \Omega_y - \mu \frac{\partial L_y}{\partial \Omega_y}$$
$$\Omega_d \leftarrow \Omega_d - \mu \frac{\partial L_d}{\partial \Omega_d}$$

Il parametro $\alpha$ è definito in modo dinamico durante l'allenamento per controllare l'entità dell'inversione del gradiente. La funzione che definisce $\alpha$ è:

$$\alpha = \frac{2}{1 + \exp(-10 \cdot p)} - 1$$

dove $p$ è una variabile che cresce durante l'allenamento (in funzione dell'epoca e del batch). Questa funzione permette a $\alpha$ di partire da valori molto bassi all'inizio dell'allenamento e di aumentare gradualmente man mano che l'allenamento procede: l'obiettivo di questo approccio è ridurre il rumore iniziale nel gradiente, che potrebbe essere troppo grande e instabile all'inizio, specialmente quando le feature apprese dal modello sono ancora poco informative [Ganin et al., 2016].

Per la **predizione** delle transizioni di fase del dominio target, dopo l'allenamento, viene utilizzato il feature extractor e il predittore di label solo sul dominio target, senza usare il layer di predittore di dominio.

[1] Nelle formule citate, $\Omega_f, \Omega_y, \Omega_d$ rappresentano i parametri dei tre layer.

## 2 Generazione dei Dati

Per lo studio delle transizioni di fase nel modello di Ising bidimensionale in presenza di campo magnetico locale randomico, è stata implementata una procedura di simulazione basata su *metodi Monte Carlo*.

Il modello di Ising bidimensionale considerato consiste in una griglia quadrata di dimensione $N \times N$, dove ogni sito della griglia rappresenta uno spin $S_i$ che può assumere valori di +1 o −1. L'energia del sistema è data dalla seguente espressione:

$$E = -J \sum_{\langle i,j \rangle} S_i S_j - \sum_i h_i S_i$$

dove $J$ è la costante di accoppiamento tra spin adiacenti ($\langle i, j \rangle$ denota i siti adiacenti), e $h_i$ rappresenta il campo magnetico locale randomico applicato al sito $i$.

La generazione dei dati è avvenuta tramite un algoritmo di *Gibbs Sampling*. L'algoritmo procede nel seguente modo:

1. **Inizializzazione**: Si assegna inizialmente alla configurazione di spin $\{S_i\}$ uno stato casuale. Se si desidera generare dati con un campo magnetico non nullo, viene definito un campo magnetico locale $h_i$ per ogni sito, scelto secondo una distribuzione specifica.
2. **Fase di Burn-in**: Vengono eseguiti numerosi passi di aggiornamento per permettere al sistema di raggiungere l'equilibrio termodinamico.
3. **Passi di campionamento**: In ogni passo, viene selezionato casualmente uno spin sulla griglia e si calcola il cambiamento di energia ($\Delta E$) se quel particolare spin venisse invertito: se $\Delta E < 0$, l'inversione viene accettata automaticamente. Altrimenti, l'inversione viene accettata con una probabilità $e^{-\Delta E/T}$ secondo il *criterio di Metropolis*.
4. **Campionamento delle Configurazioni**: Dopo aver raggiunto l'equilibrio, si raccolgono le configurazioni di spin a intervalli regolari: questo aiuta a ridurre la correlazione tra i campioni consecutivi, garantendo una migliore rappresentazione della distribuzione di Boltzmann. Le configurazioni raccolte costituiscono il dataset utilizzato per l'addestramento e la validazione dei modelli.
5. **Ripetizione per Diverse Temperature e Run**: Il processo di burn-in e campionamento viene ripetuto per diverse temperature e per un numero definito di run indipendenti.

Sono stati utilizzati i seguenti parametri:

- La costante di accoppiamento è $J = 1$
- La dimensione del reticolo è $N = 20$
- Sono stati scelti 25 valori della temperatura equispaziati tra i valori $0.2\ J < T < 5.0\ J$
- I campi $h_i$ vengono generati casualmente per ogni sito secondo una doppia distribuzione gaussiana:

$$P(h_i) = \frac{1}{2\sqrt{2\pi\sigma^2}} \left[ \exp\left( -\frac{(h_i - h_0)^2}{2\sigma^2} \right) + \exp\left( -\frac{(h_i + h_0)^2}{2\sigma^2} \right) \right] \tag{1}$$

  Il parametro $h_0$ è il modulo massimo del campo magnetico: per questa analisi sono stati scelti i valori $h_0 = \{0.5, 1.0, 1.5\}$. La varianza $\sigma^2$ è stata scelta nel limite $\sigma^2 \to 0$, cioè facendo tendere la distribuzione a una somma di due delta di Dirac: in questa approssimazione le transizioni di fase dagli stati ordinati a quelli disordinati sono considerabili continue [Crokidakis, 2009]. Per questa analisi è stato scelto il valore della deviazione standard $\sigma = 0.2$.

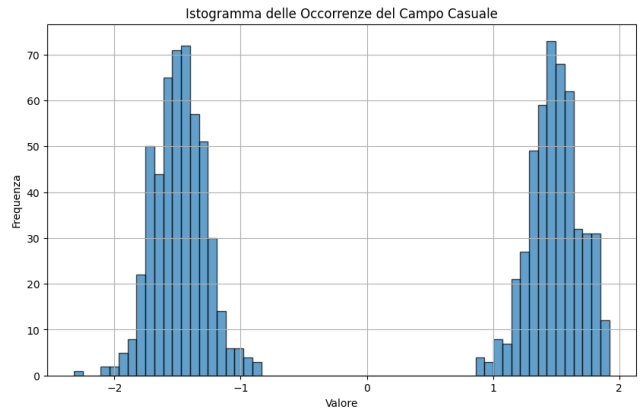



**Figura 1.** Esempio di istogramma delle occorrenze della variabile aleatoria $h_i$ descritta dall'equazione (1): $h_0 = 1.5$ , $\sigma = 0.2$

Per ciascun valore della temperatura, sono state eseguite 5000 simulazioni per garantire una copertura statistica adeguata delle configurazioni di spin per ogni dataset.

Nel caso di assenza di campo magnetico è possibile ottenere il diagramma di fase del modello di Ising appena descritto considerando la temperatura critica $T_c \simeq 2.269$ [Aviles et al., 2023]: si riportano in Figura 2 le configurazioni degli spin nelle fasi ordinate, critiche e disordinate nel caso di campo magnetico esterno nullo.

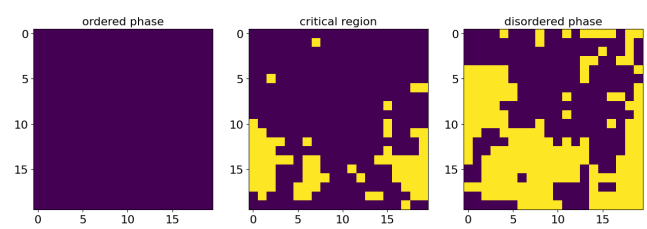



**Figura 2.** Configurazioni degli spin nelle fasi ordinate, critiche e disordinate del modello di Ising con $h_0 = 0$

## 3 Risultati

Di seguito sono riportati i risultati ottenuti analizzando le prestazioni di due modelli:

- Una CNN composta da due blocchi convoluzionali con 32 e 64 filtri rispettivamente, ciascuno con *kernel* di dimensione $3 \times 3$ e *padding* pari a 1, seguiti da funzioni di attivazione *ReLU* e layer di *Max Pooling* con *kernel* di dimensione $2 \times 2$. La fase di estrazione delle feature è completata da un layer *fully-connected* con 128 neuroni e attivazione *ReLU*, seguito da un secondo layer *fully-connected* il cui numero di neuroni è uguale al numero delle label. L'output finale è elaborato tramite una funzione di *LogSoftmax*.

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1            [-1, 32, 20, 20]             320
              ReLU-2            [-1, 32, 20, 20]               0
            Conv2d-3            [-1, 64, 20, 20]          18,496
         MaxPool2d-4            [-1, 64, 10, 10]               0
            Conv2d-5            [-1, 64, 10, 10]          36,928
              ReLU-6            [-1, 64, 10, 10]               0
            Conv2d-7            [-1, 64, 10, 10]          36,928
              ReLU-8            [-1, 64, 10, 10]               0
         MaxPool2d-9              [-1, 64, 5, 5]               0
           Flatten-10                  [-1, 1600]               0
            Linear-11                   [-1, 128]         204,928
              ReLU-12                   [-1, 128]               0
            Linear-13                     [-1, 2]             258
       LogSoftmax-14                     [-1, 2]               0
================================================================
Total params: 297,858
Trainable params: 297,858
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.66
Params size (MB): 1.14
Estimated Total Size (MB): 1.80
----------------------------------------------------------------
```

- Una DANN con estrattore di feature convoluzionale composto da tre strati con 64 filtri, *kernel* $3 \times 3$, *padding* di 1 e funzioni di attivazione *ReLU*, seguiti da layer di *Max Pooling* con *kernel* $2 \times 2$. Le feature estratte vengono passate a un layer *fully-connected* con 128 neuroni e attivazione *ReLU*. Il modello ha dunque due classificatori: uno per la predizione delle label e uno per il dominio: entrambi hanno due *layer fully-connected*. L'output è elaborato in entrmabi i casi tramite *LogSoftmax*.

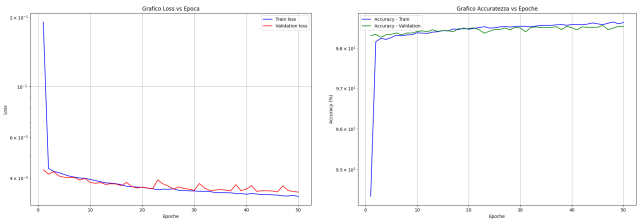
```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1            [-1, 32, 20, 20]             320
              ReLU-2            [-1, 32, 20, 20]               0
            Conv2d-3            [-1, 64, 20, 20]          18,496
              ReLU-4            [-1, 64, 20, 20]               0
         MaxPool2d-5            [-1, 64, 10, 10]               0
            Conv2d-6            [-1, 64, 10, 10]          36,928
              ReLU-7            [-1, 64, 10, 10]               0
            Conv2d-8            [-1, 64, 10, 10]          36,928
              ReLU-9            [-1, 64, 10, 10]               0
        MaxPool2d-10              [-1, 64, 5, 5]               0
           Flatten-11                  [-1, 1600]               0
            Linear-12                   [-1, 128]         204,928
              ReLU-13                   [-1, 128]               0
           Dropout-14                   [-1, 128]               0
            Linear-15                   [-1, 256]          33,024
              ReLU-16                   [-1, 256]               0
           Dropout-17                   [-1, 256]               0
            Linear-18                     [-1, 2]             514
       LogSoftmax-19                     [-1, 2]               0
================================================================
Total params: 331,138
Trainable params: 331,138
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.86
Params size (MB): 1.26
Estimated Total Size (MB): 2.13
----------------------------------------------------------------
```

L'allenamento della CNN utilizza *NLLLoss* come funzione di perdita e l'ottimizzatore *Adam* con un learning rate di $lr = 10^{-5}$: è stato preferito rispetto ad altri per la sua capacità di adattare dinamicamente il learning rate, garantendo una convergenza stabile e veloce rispetto a un normale *SGD*. La metrica di accuratezza è monitorata sia sul training che sulla validazione, salvando il miglior modello in base alle performance sui dati di validazione (del dominio sorgente).

L'allenamento della DANN utilizza invece due funzioni di perdita *NLLLoss* e due ottimizzatori *SGD* con learning rate $lr = 10^{-4}$ e momentum $m = 0.9$ per ciascun classficiatore. Viene monitorata l'accuratezza sia delle label che del dominio, e il miglior modello viene salvato in base alla performance sui dati di validazione.

### *3.1 Rete Neurale Convoluzionale (CNN)*

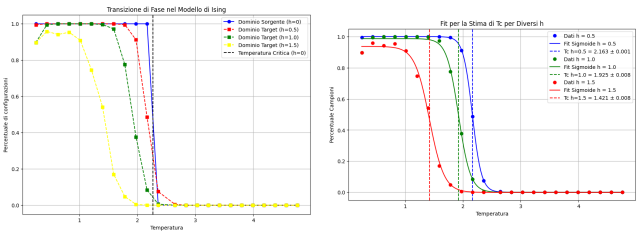
I dati appartenenti al dominio di sorgente e target sono stati elaborati dalla rete tramite dei *data loader* con batch size pari a 128. I dati del dominio sorgente sono stati divisi a loro volta nei set di *train* (75 %) e *validation* (25 %). Si visualizzano in Figura 3 le loss e dell'accuratezza in funzione del numero di epoca.




(1) (2)

**Figura 3.** Loss (1) e accuratezza (2) in funzione del numero di epoche.

In Figura 4 sono riportati i grafici dei diagrammi di fase ottenuti stimando le label per il dominio target tramite il modello. Osservando l'andamento delle curve, si è deciso di stimare la temperatura critica $T_c(h)$ tramite un fit con una curva sigmoide. I risultati del fit sono mostrati in Tabella 1 :




(1) (2)

**Figura 4.** Diagramma di fase dei vari modelli di Ising (1) e stima della temperatura critica $T_c$ tramite fit con curva sigmoide (2) .

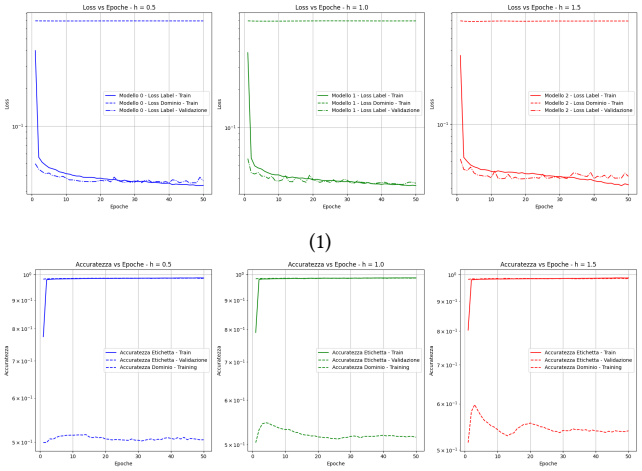
| $h_0$ | $T_c$ |
|---|---|
| 0.5 | $(2.163 \pm 0.001)\ J$ |
| 1.0 | $(1.925 \pm 0.008)\ J$ |
| 1.5 | $(1.421 \pm 0.008)\ J$ |

**Tabella 1**
Valori di $T_c$ al variare di $h_0$ utilizzando il modello CNN.

### *3.2 Domain Adversarial Neural Network (DANN)*

I dati appartenenti al dominio di sorgente e target sono stati elaborati dalla rete tramite dei *data loader* con batch size pari a 128. I dati del dominio sorgente sono stati divisi a loro volta nei set di

*train* (75 %) e *validation* (25 %). Si visualizzano in Figura 5 le loss e le accuratezze in funzione del numero di epoca per ognuno dei domini target analizzati:

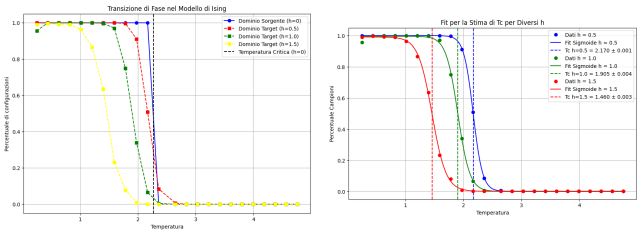



(1)

(2)

**Figura 5.** Loss (1) e accuratezza (2) in funzione del numero di epoche.

Vengono inoltre riportati in Figura 6 e in Tabella 2 i grafici dei diagrammi di fase e le stime della temperatura critica $T_c(h)$:




(1) (2)

**Figura 6.** Diagramma di fase dei vari modelli di Ising (1) e stima della temperatura critica $T_c$ tramite fit con curva sigmoide (2).

| $h_0$ | $T_c$ |
|---|---|
| 0.5 | $(2.170 \pm 0.001)\ J$ |
| 1.0 | $(1.905 \pm 0.004)\ J$ |
| 1.5 | $(1.460 \pm 0.003)\ J$ |

**Tabella 2**
Valori di $T_c$ al variare di $h_0$ utilizzando il modello DANN.

### *3.3 Commento*

Dall'analisi condotta, ciò che si può affermare è che, come confermato negli articoli Crokidakis, 2009 e Huembeli et al., 2018, la temperatura critica diminuisce all'aumentare dell'intensità del campo magnetico.

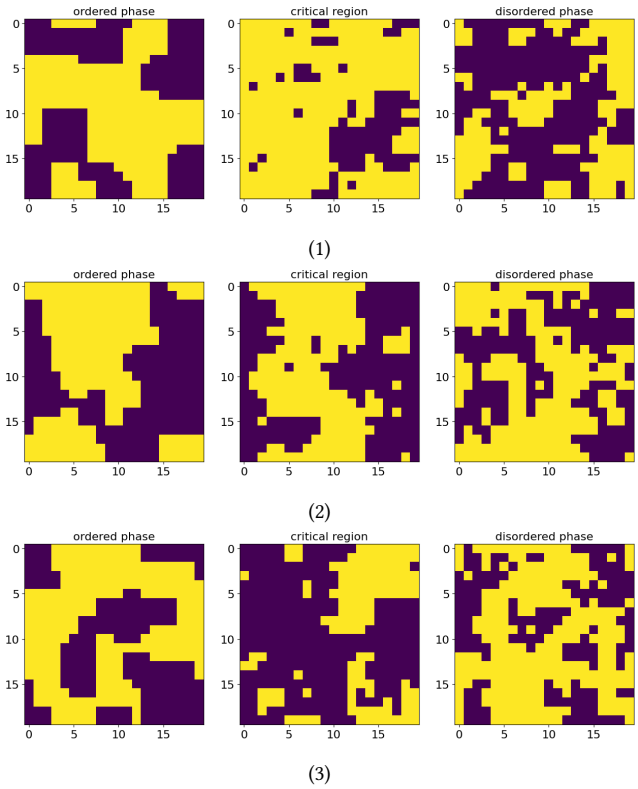
Da come si può osservare in Tabella 3, la CNN ha fornito una stima della temperatura critica $T_c$ molto vicina ai risultati ottenuti con la DANN per i vari valori del campo magnetico $h$, come evidenziato nelle tabelle : come già osservato nell'articolo Huembeli et al., 2018, una rete convoluzionale può imparare i parametri d'ordine del modello di Ising senza la necessità di applicare alcun *fine-tuning* per il *transfer learning*.

Tuttavia, l'approccio della DANN ha il vantaggio di sfruttare la conoscenza del dominio di sorgente per adattarsi meglio al dominio target, in questo caso con campo magnetico esterno: l'inclusione del classificatore di dominio ha permesso di generalizzare meglio le feature in presenza di campi magnetici disordinati, fornendo una stima più accurata delle transizioni di fase rispetto ai risultati riportati nell'articolo Crokidakis, 2009.

| $h_0$ | $T_c$ **(CNN)** | $T_c$ **(DANN)** |
|---|---|---|
| 0.5 | $(2.163 \pm 0.001)\ J$ | $(2.170 \pm 0.001)\ J$ |
| 1.0 | $(1.925 \pm 0.008)\ J$ | $(1.905 \pm 0.004)\ J$ |
| 1.5 | $(1.421 \pm 0.008)\ J$ | $(1.460 \pm 0.003)\ J$ |

**Tabella 3**
Valori di $T_c$ al variare di $h_0$ utilizzando i modelli CNN e DANN.

In Figura 7 vengono mostrate le configurazioni degli spin dei vari modelli di Ising, nelle loro fasi ordinate, critiche e disordinate.




(1)

(2)

(3)

**Figura 7.** Configurazioni degli spin nelle fasi ordinate, critiche e disordinate del modello di Ising con $h_0 = 0.5$ (1), $h_0 = 1.0$ (2) e $h_0 = 1.5$ (3).

## Riferimenti bibliografici

Aviles, L A e R P Gammag (lug. 2023). "Critical temperature in 2D square Ising model with anisotropic coupling constant using Kramers-Wannier duality". In: *Journal of Physics: Conference Series* 2543.1, p. 012006. DOI: 10.1088/1742-6596/2543/1/012006. URL: https://dx.doi.org/10.1088/1742-6596/2543/1/012006.

Crokidakis, Nuno (feb. 2009). "First-order phase transition in a 2D random-field Ising model with conflicting dynamics". In: *Journal of Statistical Mechanics: Theory and Experiment* 2009.02. DOI: 10.1088/1742-5468/2009/02/P02058. URL: https://dx.doi.org/10.1088/1742-5468/2009/02/P02058.

Ganin, Yaroslav et al. (2016). "Domain-Adversarial Training of Neural Networks". In: *Journal of Machine Learning Research* 17.59. URL: http://jmlr.org/papers/v17/15-239.html.

Huembeli, Patrick, Alexandre Dauphin e Peter Wittek (apr. 2018). "Identifying quantum phase transitions with adversarial neural networks". In: *Physical Review B* 97.13. ISSN: 2469-9969. DOI: 10.1103/physrevb.97.134109. URL: http://dx.doi.org/10.1103/PhysRevB.97.134109.